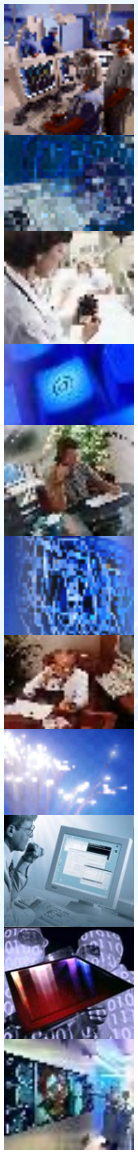
ISIS Ingénieur Santé numérique

UC Institut National Universitaire Champollion



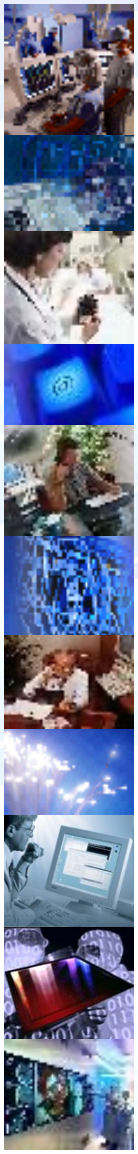
UE Technologies du web
le langage javascript

Jean-Marie Pécatte
jean-marie.pecatte@iut-tlse3.fr

JS

Avec l'aide des supports de cours de S. Trouilhet en DUT MMI et Elyes Lamine à ISIS3A

Généralités



- 1995 – Brendan Eich de Netscape Communications (Netscape > Firefox de Mozilla)
- Langage de programmation de script
- Principalement utilisé pour les pages web (front end) mais aussi coté serveur (avec nodejs en backend)
- Langage standardisé => ECMAScript
- Edition 5 (ES5) [décembre 2009]
- Edition 6 (ES6) [juin 2015]
- Changement du processus de normalisation avec une version chaque année
  - Edition 7 ➔ ES2016 (juin 2016)
  - Edition 15 ➔ ES2024 (juin 2024)

Généralités

- Le langage est interprété par un "moteur javascript" qui implémente un certain niveau de la norme
- **SpiderMonkey** est le moteur de Firefox
- **V8** (Google) est le moteur de Chrome, Microsoft Edge (Chromium) Brave, Opera, Vivaldi *(mais aussi de NodeJS coté serveur)*
- **Chakra** est le moteur d'EDGE -> abandonné
- **JavascriptCore** pour Safari
- …

  ➔moteurs différents => pgm pas toujours portable d'un navigateur à un autre…

  ➔ moteur et version ECMAscript ?

  https://compat-table.github.io/compat-table/esnext/

Généralités

Classement des langages de programmation *(12/2025)*

- TIOBE *(moteurs de recherche)* : javascript en 6ème position
- REDMONK *(github, stack overflow)* : javascript en 1ère position
- IEEE Spectrum *(classement le plus complet)* : javascript en 6ème position

- Langage très utilisé => nombreux framework
  - Angular, React, Vue, Meteor, …
- Très grande richesse de modules disponibles
- Enquête développeurs JS :

  https://2024.stateofjs.com/en-US

Eléments du langage

Eléments du langage : types de données

- Boolean → true et false
  - valeurs équivalentes à faux :

    false, null, undefined, '', 0, NaN (Not a Number)
- Number
  - valeur numérique sur 64 bits (entiers et réels)

    10 -3 1e2 3.14 0723 (octal) 0x723 (hexadécimal)
- String
  - chaîne de caractères (caractère codé sur 16 bits)
  - entre " " ou ' '

    "bonjour" 'bonjour' "\t"

Eléments du langage : Interpolation ou template de chaîne

- L'interpolation (ou template) de chaîne permet d'introduire directement des variables (ou des expressions) dans une chaine ; elle permet aussi d'écrire une chaine sur plusieurs lignes.
- Pour cela il faut utiliser des ` ` *(backquote/backtick)* et introduire les expressions avec cette syntaxe ${ expr }

```
let prenom = 'Jean'
let age = 37
let message = `Bonjour, mon nom est ${ prenom }.
      j'aurai ${ age + 1 } ans le mois prochain `
```

```
Bonjour, mon nom est Jean.
      j'aurai 38 ans le mois prochain
```

ISIS Ingénieur Santé numérique

Institut National Universitaire Champollion

Eléments du langage : opérateurs

Les **opérateurs** du langage :

- arithmétiques + - * / % *(modulo)*
- comparaison < <= > >= == !=
- logiques && (et) || (ou) ! (négation)
- opérateur typeof pour connaître le type d'une variable :
  'number' 'string' 'boolean' 'undefined' 'function' 'object'
- concaténation : + *(même opérateur que l'addition)*

"salut" + "monde" → **"salutmonde"**

Eléments du langage : Opérateur ternaire

- L'opérateur ternaire permet d'écrire une instruction conditionnelle sous forme d'expression :

```
if (a > b) {
   c = 1
} else {
   c = -1
}
```

```
c = ( a > b ) ? 1 : -1
```

Eléments du langage : variables

- Déclarations de variables
  - ~~**var variable1**~~ : la portée de la variable est la fonction ou l'espace global ; ce type de variable peut être utilisée avant d'être déclarée
  - **let** variable2 : la portée est limitée au bloc de la déclaration
  - **const** variable3 : l'initialisation est obligatoire et ne peut être modifiée ; la portée est limitée au bloc de la déclaration
  - utilisation d'une **variable non déclarée** → déclaration automatique d'une variable globale avec le mot clé var -> déconseillé -> interdit en mode « strict » ES6+

Eléments du langage : variables

Diapo facultative : utile pour comprendre le fonctionnement de var en JS

Exemples

```
vUn = 12;
var vUn = 7;
```

pas d'erreur car en JS la déclaration var est remontée (hosting)

le code exécuté est celui-ci

```
var vUn;
vUn = 12;
vUn = 7
```

```
vUn = 12;
let vUn = 7;
```

erreur "not defined" car la déclaration avec let n'est pas remontée

Eléments du langage : variables

Diapo facultative : utile pour comprendre le fonctionnement de var en JS

Exemples

```
vDeux = 4
function secon() {
        vDeux = 12;
}
console.log(vDeux)
secon();
console.log(vDeux);
```

→pas d'erreur car une fonction peut accéder aux variables globales

→ 4

→ 12 modifié par l'appel de la fct

Eléments du langage : variables

Diapo facultative : utile pour comprendre le fonctionnement de var en JS

Exemples

```
vDeux = 4
function secon() {
    vDeux = 12;
    if (1) {
        var vDeux = 18;
        let vUn = 12
    }
    console.log(vDeux);
    console.log(vUn);
}
secon();
console.log(vUn);
console.log(vDeux);
```

→ 18 modifié par le bloc du if

→ erreur "not defined" car vUn n'est définie que dans le bloc du if

→ erreur "not defined"

→4 **pourquoi ???**

Eléments du langage : variables

Diapo facultative : utile pour comprendre le fonctionnement de var en JS

En fait le code est **il y a 2 variables vDeux !!**

```
var vDeux = 4                 →variable vDeux globale
function secon() {
    var vDeux = 12;           →variable vDeux locale à la fonction
    if (1) {
        var vDeux = 18;
        let vUn = 12
    }
    console.log(vDeux);       → variable locale
}
secon();
console.log(vDeux);           → variable globale
```

Eléments du langage : instructions

Instructions

- Instruction simple : terminée par un point virgule (facultatif)
- Séquence d'instructions : Blocs d'instructions délimitées par des accolades **{ }**
- Structure de sélection :

```
if ( condition )   {
        ...
  }
```

```
if ( condition ) {
        ...
}  else {
 ...
 }
```

Eléments du langage : instructions

- Exemple : afficher la mention d'une note (P, AB, B, TB)

```
if ( note >= 16 ) {
 console.log("Très bien")
} else {
      if ( note >= 14 ) {
            console.log("Bien")
      } else {
            if ( note >= 12 ) {
             console.log("Assez bien")
            } else {
                  console.log("Passable")
            }
      }
}
```

Eléments du langage : instructions

- Structures de répétition :

```
while ( condition ) {
        ...
}
```

```
do {
        ...
} while ( condition ) ;
```

```
for (let i=1; i<=10; i=i+1 ) {
    ...
}
```

Eléments du langage : instructions

- Exemple: calculer la somme des 10 premiers nombres entiers positifs

```
let somme = 0 ;
let nb = 1 ;
while (nombre <= 10) {
 somme = somme + nombre ;
 nb = nb + 1 ;
}
```

```
let somme = 0 ;
for ( let nb= 1 ; nb <= 10 ; nb++ ) {
 somme += nb ;
}
```

Eléments du langage : instructions

- Structures de répétition :

```
for ( let p in objet )  {
  ...
}
```

```
for ( let val of objet )  {
  ...
}
```

Semblable ‘for each’ java

```
let liste = [12,9,20,14];
for (let i in liste) {
      console.log(i);
}
for (let val of liste) {
      console.log(val);
}
```

→ 0 1 2 3

→ 12 9 20 14

Eléments du langage : instructions

- Branchement multiple :

```
switch ( expression ) {
    case label1 :      …      ; break ;
    case label2 :      …      ; break ;
    …
    default : … ; break ;
}
```

Eléments du langage : tableaux

Les tableaux

- Tableau = regroupement de plusieurs valeurs de même type (ensemble de notes, de produits, …)
- Déclaration

```
let tableau = [ ] // déclaration d'un tableau vide
let tNotes = [12,20,11] // déclaration d'un tableau
 contenant 3 notes ; les indices des cases sont 0,1,2

alert( tNotes[1] )  // affiche 20

tNotes[1] = 34 // modifie la valeur de la case d'indice 1
alert( tNotes[1] ) // affiche 34

alert( tNotes.length) // affiche 3, la taille du tableau
```

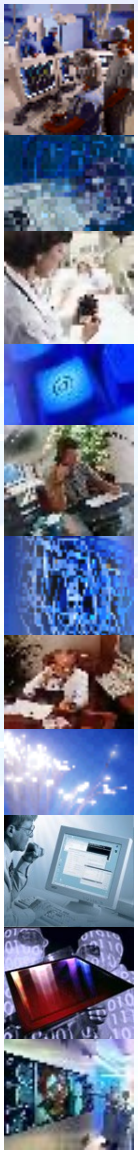
Eléments du langage : tableaux

- Exemple : chercher la valeur la plus petite dans un tableau contenant les valeurs 9,11,2,21,16

```
let tabInt = [9, 11, 2, 21, 16]
```

```
let petite = tabInt[0]
for ( let i = 1 ; i < tabInt.length ; i = i + 1 ) {
  if ( tabInt[i] < petite ) { petite = tabInt[i] }
}
```

Eléments du langage : objet littéral



- *Objet littéral :*

  avec les { }, ensemble de couple clé : valeur

```
let personne = {
 nom : "Dupont",
 prenom : "Jean",
 age : 45
}
```

→ *accès à une propriété : personne.nom*

→ *une propriété peut elle-même être un objet littéral ou un tableau*

Langage : Déstructuration / propagation

- déstructurer un tableau ou un objet : permet d'extraire des éléments de tableau ou d'objets dans plusieurs variables :

→ stocker les 2 valeurs d'un tableau dans 2 variables

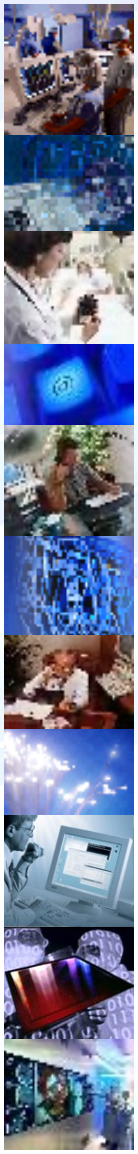
let [un, deux] = [1, 2] ; → un = 1 deux =2

→ stocker les 3 propriétés d'un objet dans trois variables en une seule instruction

let { vnom, vprenom, vage } = personne;

→ vnom = "Dupont" vprenom = "Jean" vage= 45

Langage : déstructuration / propagation



Mais il est aussi possible de :

→ échanger les valeurs de 2 variables

[un, deux] = [deux, un];

→ créer une variable avec le reste du tableau

let [un, …suite] = [1, 2, 3, 4];

→ un = 1 suite = [2, 3, 4]

→ Créer des variables que pour certains éléments :

let [un] = [1,2,3,4];

let [, deux, , quatre] = [1,2,3,4]

ISIS Ingénieur Santé numérique

Institut National Universitaire Champollion

Langage : déstructuration / propagation

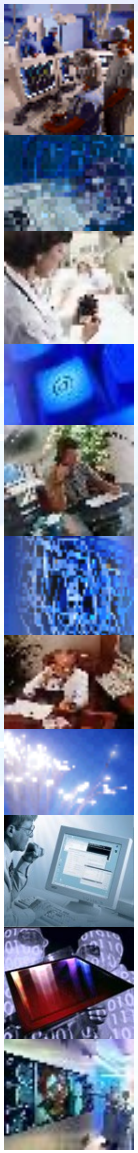
- propager plusieurs bouts de tableau en un seul :

let un = [1,2];
let deux = [3, 4];
let trois = **[** 0, …un, …deux, 5 **]**; → [0,1,2,3,4,5]

- créer une copie d'un tableau ou d'un objet

let tab1 = [3, 4];
let tab2 = tab1; → tab2 n'est pas une copie
tab2[0] = 8 → tab1[0] est aussi modifié et vaut 8
let tab3 = **[…**tab1**]** → tab3 est une copie
tab3[0] = 12 → tab1[0] n'est pas modifié et vaut tjs 8

Eléments du langage : objets



En javascript, il y a de très nombreux objets prédéfinis.

Utilisation : notation pointée pour accéder aux propriétés et méthodes d'un objet

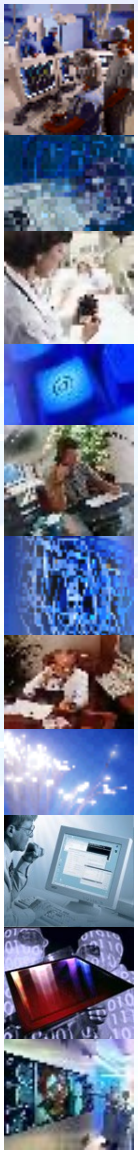
- le type String est un objet
  - let str = new String("bonjour") // ou var str = "bonjour"
  - str.length → longueur
  - str.split(*sep*) → décompose str en plusieurs sous-chaines séparées par *sep*
  - str.indexOf(*ch*) →renvoie la 1ère position de *ch* dans la chaine *str* ; -1 si *ch* n'est pas dans *str*
  - str.substr(*pos, nb*) →renvoie une sous-chaine de *str* à partir de la position *pos* et contenant *nb* caractères

Eléments du langage : objets

- le type Array est un objet
  - let tab = new Array(11,7,4,6) // ou tab = [11,7,4,6]
  - tab.length → nombre d'éléments
  - tab.push(elt) → ajout *elt* à la fin du tableau
  - tab.pop() → supprime le 1er elt du tableau
  - tab.reverse() → renverse le tableau
  - tab.sort() → tri le tableau
  - tab.indexOf(val) → position de la 1ère val, -1 sinon
  - tab.join() → concatène ts les elts en une chaine
  - tab.forEach() → parcourir le tableau

→ Attention : let tab = liste; ne crée pas une copie du tableau

Eléments du langage : fonctions



Les fonctions

- Une fonction regroupe un ensemble d'instructions que l'on peut/veut utiliser plusieurs fois dans un programme
- Déclaration

```
function mafonction( paramètres ) {
    …
    return … ;
}
```

- Appel

```
mafonction(paramètres)
```

➔ en javascript, il est possible d'appeler une fonction avant sa définition

Eléments du langage : fonctions

Exemple

- somme de 2 nombres

```
function somme(nb1, nb2) {
    let resultat
    resultat = nb1 + nb2;
    return resultat;
}
let res = somme(12,18);
```

→ variable locale
→ algo de la fonction
→ valeur de retour
→ res = 30

→ version plus synthétique

```
function somme(nb1, nb2) {
    return nb1 + nb2;
}
let res = somme(12,18);
```

Eléments du langage : fonctions

- En javascript, les fonctions sont des objets ; il est possible de stocker une fonction dans une variable

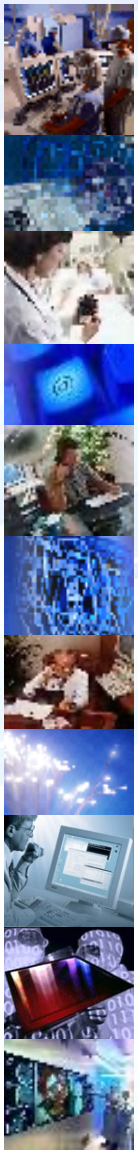
```
let addition = function(n1, n2) {
    return n1 + n2;
};
let resultat = addition(4, 7);
```

→ res = 11

- Donc une fonction peut avoir en paramètre une autre fonction

```
function surprise(mafonction) {
    return mafonction();
}
```

Eléments du langage : fonctions



```
function bonjour() { console.log("bonjour"); }

function aurevoir() { console.log("aurevoir"); }
```

```
surprise(bonjour);
```

→ "bonjour"

```
surprise(aurevoir);
```

→ "aurevoir"

```
surprise( function() {
    console.log("coucou") }
);
```

→ "coucou"

Eléments du langage : fonctions

- Rappels fonctions :
  - avec des paramètres (non typés et obligatoires)
  - une valeur de retour non typée
  - l'instruction return définie la valeur de retour

```
function nomcomplet(prenom, nom) {
   return `${prenom} ${nom}`;
}

let result1 = nomcomplet("Jean"); // erreur
let result2 = nomcomplet("Jean", undefined); // erreur
let result3 = nomcomplet("Jean", "Dupont", "Dr."); // erreur
let result4 = nomcomplet("Jean", "Dupont"); // ok Jean Dupont
```

Eléments du langage : fonctions

- paramètre avec valeur pas défaut :

```
function nomcomplet(prenom, nom = "Durand") {
      return `${prenom} ${nom}`;
}

let result1 =
   nomcomplet("Jean"); // ok Jean Durand
let result2 =
   nomcomplet("Jean", undefined); // ok Jean Durand
let result3 =
   nomcomplet("Jean", "Dupont", "Dr."); // erreur
let result4 =
   nomcomplet("Jean", "Dupont"); // ok Jean Dupont
```

Elts du langage : fonctions fléchées

- Une expression de fonction fléchée permet d'obtenir un code plus synthétique (plus court) qu'une fonction
- Une fonction fléchée n'a pas de **this** spécifique, c'est-à-dire pas de contexte spécifique ; si le mot clé this est utilisé, il fera référence au contexte parent (closure)
- Une fonction fléchée est anonyme
- Etant anonymes, les fonctions fléchées sont :
  - stockées dans des variables
  - utilisées en paramètre de fonctions

Elts du langage : fonctions fléchées

- Syntaxe d'une fonction fléchée :
  - (param1, param 2) => { instructions }

```
(a, b) => {
    let c = a + b
    return c
}
```

  - (param1, param 2) => { return expression }

```
(a, b) => { return a + b }
```

peut être abrégée sans les { } et return

(param1, param 2) => expression

```
(a, b) => a + b
```

Elts du langage : fonctions fléchées

- Syntaxe d'une fonction fléchée :
  - (param1) => { corps de la fonction } // un seul paramètre

    peut être abrégé sans les () autour du paramètre

    param1 => { corps de la fonction }

```
function carre(x) { return x * x }
```

```
const carre = (x) => { return x * x }
```

```
const carre = (x) => x * x
```

```
const carre = x => x * x
```

Elts du langage : fonctions flèches

- Exemple 1 : stocker une fonction flèche dans une variable

```
const somme = (a, b) => { return a + b }
// utilisation comme une fonction « classique »
let resultat = somme(5, 10)
```

- Exemple 2 : utiliser une fonction flèche en paramètre

on suppose que la méthode truc accepte une fonction en paramètre qui compare deux nombres et renvoie vrai si le 1er est plus grand que le 2ème , faux sinon

```
liste.truc ( function (n1, n2 ) {
    if ( n1 > n2 ) {
      return true
    } else {
      return false
    }
})
```

```
liste.truc((n1, n2) => {
      return n1 > n2
}})
```

ISIS Ingénieur Santé numérique

Institut National Universitaire Champollion

Eléments du langage : closures

Diapo facultative : utile pour comprendre le fonctionnement des closures

- Une closure (ou encore fermeture) est une fonction qui se souvient de l'environnement dans lequel elle a été définie ; elle "capture" son environnement.

```
function test() {
    let texte="Bonjour";
    function saluer() {
        console.log(texte);
    }
    return saluer;
}
let hello = test();
hello();
```

→ fonction qui renvoie une fonction

→ peut utiliser les variables de la fonction qui l'englobe

→ hello contient la fonction saluer

→ execution de la fonction saluer

→ problème la var texte n'existe pas !!!

Eléments du langage : closures

Diapo facultative : utile pour comprendre le fonctionnement des closures

- Le même exemple en considérant que la fonction saluer est une closure.

```
let texte="Bonjour";
function saluer() {
    console.log(texte);
}

let hello = test();
hello();
```

→ la closure conserve son contexte de définition donc les variables de la fonction englobante

hello(); → cette fois la variable existe → "Bonjour"

Eléments du langage : closures

Diapo facultative : utile pour comprendre le fonctionnement des closures

- Autre exemple

```
function faireAddition(y)
    function add(x) {
        return x + y;
    }
  return add;
}

let add3 = faireAddition(3);
let add5 = faireAddition(5);
console.log(add3(6));
console.log(add5(6));
```

→ la closure conserve son contexte de définition donc les **variables** de la fonction englobante mais aussi les **paramètres**

→ le contexte contient y=3

→ le contexte contient y=5

→ 9

→ 11

Tableau et fonctions

- forEach : applique une fonction sur chaque elt du tableau ; le tableau n'est pas modifié.

```
array.forEach( (valeur,index) => {
    traitement
} )
```

```
let T = [8, 3, 12];
T.forEach( (v) => console.log(v) )
    → 8  3  12
```

équivalent à

```
for (v of T) { console.log( v ) }
```

Tableau et fonctions

- sort : tri les éléments du tableau

```
array.sort( (v1,v2) => résulat )
```

la fct doit avoir 2 params v1 et v2 = les 2 vals à comparer

elle doit renvoyer comme resultat une valeur

- < 0 si v1 avant v2
- > 0 si v1 après v2
- 0 si « inchangé »

```
let T = [8, 3, 12];
  T.sort( (v1,v2) => v1 < v2 ? 1 : -1)
        → [12, 8, 3] → tri décroissant
```

Tableau et fonctions

- sort : tri les éléments du tableau

cas particulier si pas de paramètre

→ tri croissant sous forme de chaine de caractères

```
let T = [8, 3, 12];
T.sort( ) --> [12, 3, 8]
```

mais pas [3, 8, 12]

→ tri croissant *(mais sous forme de string)*

Tableau et fonctions

- map: applique une fonction sur chaque elt du tableau et stocke le résultat dans le tableau renvoyé ; le tableau résultat a la même taille que le tableau T

```
array.map( (v) => nouvelle_valeur )
```

- <u>exemple</u>

```
let T = [8, 3, 12];
let T2 = T.map( (v) => v - 3 ) → [5, 0, 9]
```

équivalent à

```
let T2 = [ ];
for (let i=0; i < T.length ; i++) {
   T2[i] = T[i] - 3 ;
}
```

Tableau et fonctions

- reduce : applique la fonction sur chaque elt pour réduire le tableau à une seule valeur

```
array.reduce(
  (res, v) => nv_res, v_initiale)
```

→ res : le résultat en cours de calcu

→ v : un elt du tableau

→ v_initiale : valeur de départ de res

→ nv_res : nouvelle valeur de res

- <u>exemple</u>

```
let T = [8, 3, 12];
let somme=T.reduce((res, v) => res + v,0)
                          → 23
```

Tableau et fonctions

- reduce : applique la fonction sur chaque elt pour réduire le tableau à une seule valeur

équivalent à

```
let somme = 0;
for (let i=0; i < T.length ; i++) {
    somme = somme + T[i];
}
```

Tableau et fonctions

- filter : applique la fonction sur chaque elt pour réduire le tableau à un tableau qui ne contient que les valeurs vérifiant la condition
- Exemple

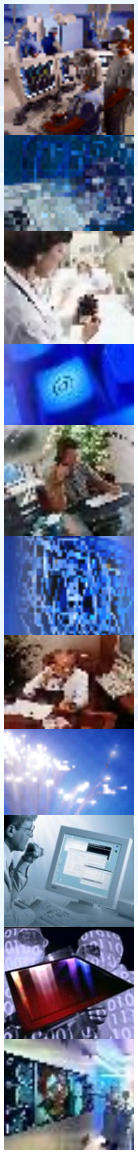
```
let T = [8, 3, 12];
let inf10 = T.filter( (v) => v < 10)
                    → [8, 3]
```

équivalent à

```
let inf10 = [ ], j = 0 ;
for (let i=0; i < T.length ; i++) {
     if ( T[i] <10 ) { Inf10[j] = T2[i];
                       j = j +1; }
   }
```

ISIS Ingénieur Santé numérique

UC Institut National Universitaire Champollion



Javascript et le web

Javascript et le web

utilisation de JS

- agir sur les propriétés d'un document HTML
- manipuler l'arbre DOM

Pour

- réaliser des applications interactives (par ex. tests et quiz)
- réaliser des pages HTML plus riches (par ex. animations,...)
- générer du code HTML selon le profil de l'utilisateur
- vérifier des formulaires avant traitement sur un serveur
- …

Javascript et le web

- **Dans le corps du document** (entre <body> </body>)

le code JavaScript est exécuté une fois lors du chargement de la page dans le navigateur

- **Dans l'entête du document** (entre <head> </head>)

Le code Javascript n'est pas exécuté, il est juste stocké dans le navigateur

- Pour le distinguer du code HTML, dans ces 2 cas, le code javascript doit être saisi entre les 2 balises <script> </script>

```
<script>
…
</script>
```

Javascript et le web

- **Dans un fichier externe (.js)**

```
<script src="script.js"></script>
```

- **Dans des balises HTML**

Permet d'executer des bouts de code javascript quand certains événements (chargement de la page, clic de souris, …) surviennent

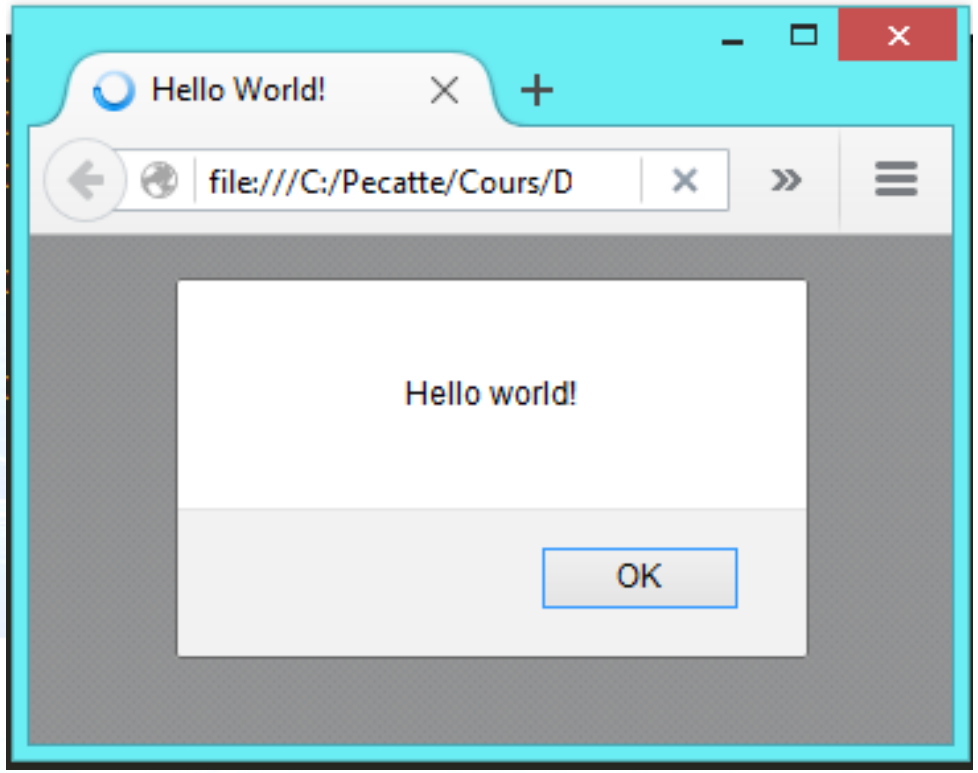
```
<a href="…" onclick="alert('Bonjour !');">lien</a>
```

Mauvaise pratique : ne pas utiliser !

Javascript et le web

```
<!DOCTYPE html>
<html>
<head>
    <title>Hello World!</title>
</head>
<body>
    <script type="text/javascript">
        alert('Hello world!');
    </script>
</body>
</html>
```

L'écran d'affichage des résultats est la fenêtre du navigateur

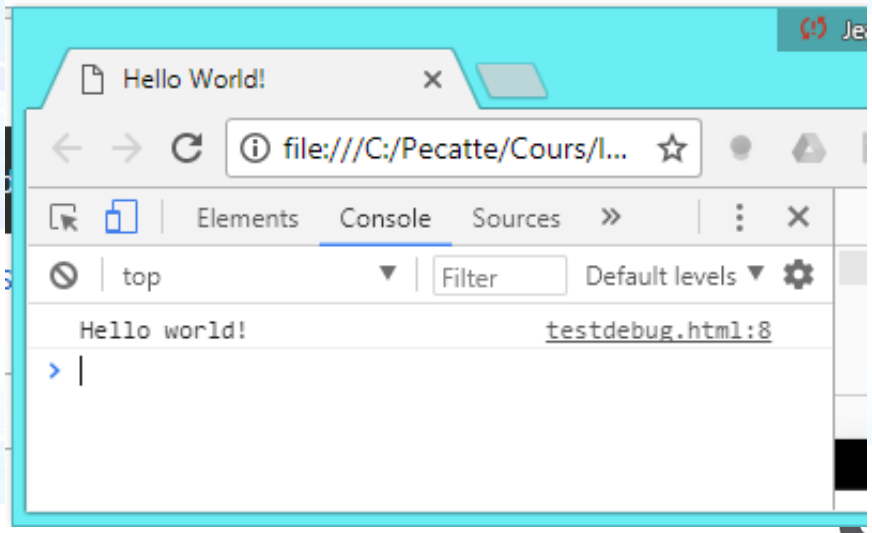


Javascript et le web

Débogage de scripts javascript

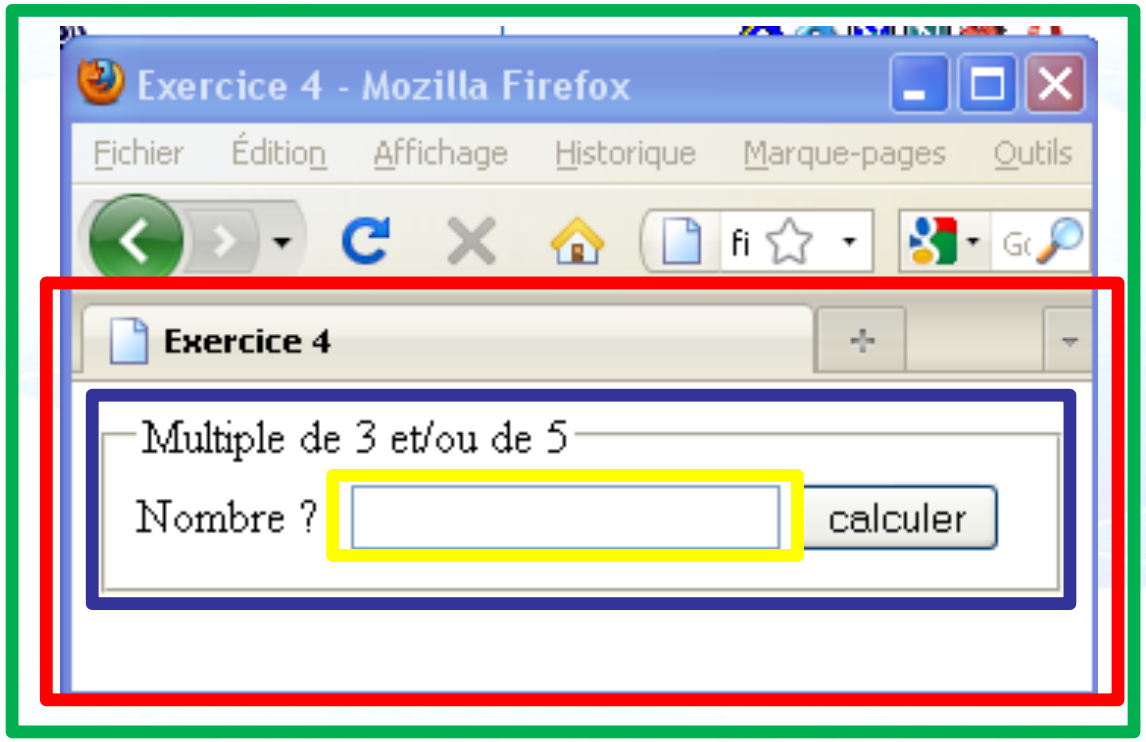
- Tous les navigateurs proposent des "outils pour les développeurs"
- L'élément le plus utilisé est la **console** ; elle permet d'y afficher des informations qui ne seront pas visibles par l'internaute
- instruction **console.log("…")**

```
...
<script>
  console.log('Hello world!');
</script>
..
```



Les objets prédéfinis

- Javascript est un langage basé sur des objets ; tous les éléments du navigateur et du document HTML sont des objets.



window

document

form

nombre

Les objets prédéfinis

- Les objets sont organisés sous la forme d'une arborescence
- Pour accéder à un objet, il faut donner le chemin d'accès depuis la racine (window)

Exemple : **window**.**document**.**form**.**nombre**

- Pour accéder à un objet du document HTML, il est aussi possible de le rechercher si l'élément à un ID en HTML.

```
<img id="img1" src="…" />
```

```
let img=document.getElementById("img1");
```

le DOM
(Document Object Model)

le DOM *(Document Object Model)*

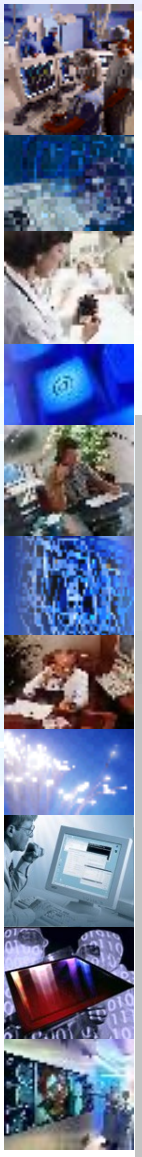
- modèle standardisé par le W3C (World Wide Web Consortium)
- représente un document sous la forme d'un arbre
- toutes les balises HTML sont donc des nœuds de l'arbre
- les feuilles sont soit des balises sans contenu, soit le texte de la page HTML
- plusieurs types de nœud :
  - Nœud de type élément (balise HTML)
  - Nœud de type texte (contenu d'un élément)
  - Nœud de type commentaire

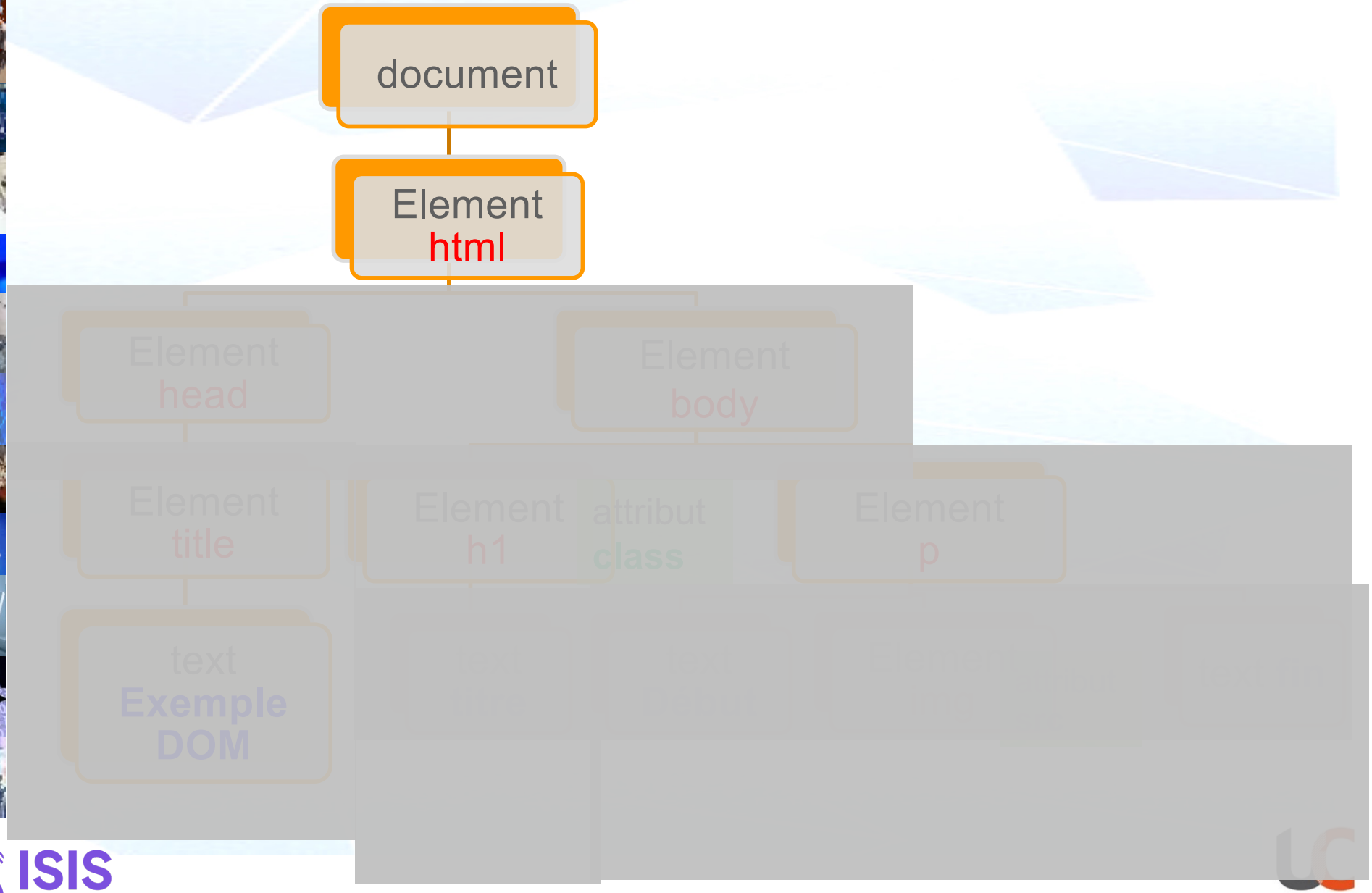
➔ Dans le DOM récent, les attributs ne sont plus des nœuds mais des propriétés.

le DOM *(Document Object Model)*

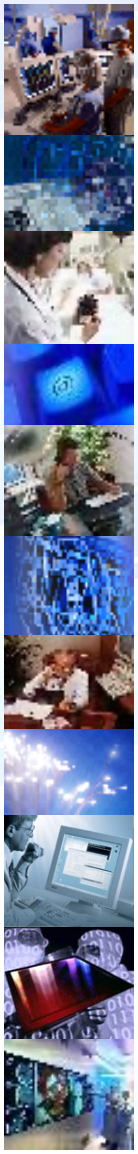
```
<!DOCTYPE html>
<html>
<head>
    <title>Exemple DOM</title>
</head>
<body>
    <h1 class="titre">Titre</h1>
    <p>Début<img src="img1.jpg"/>fin</p>
</body>
</html>
```

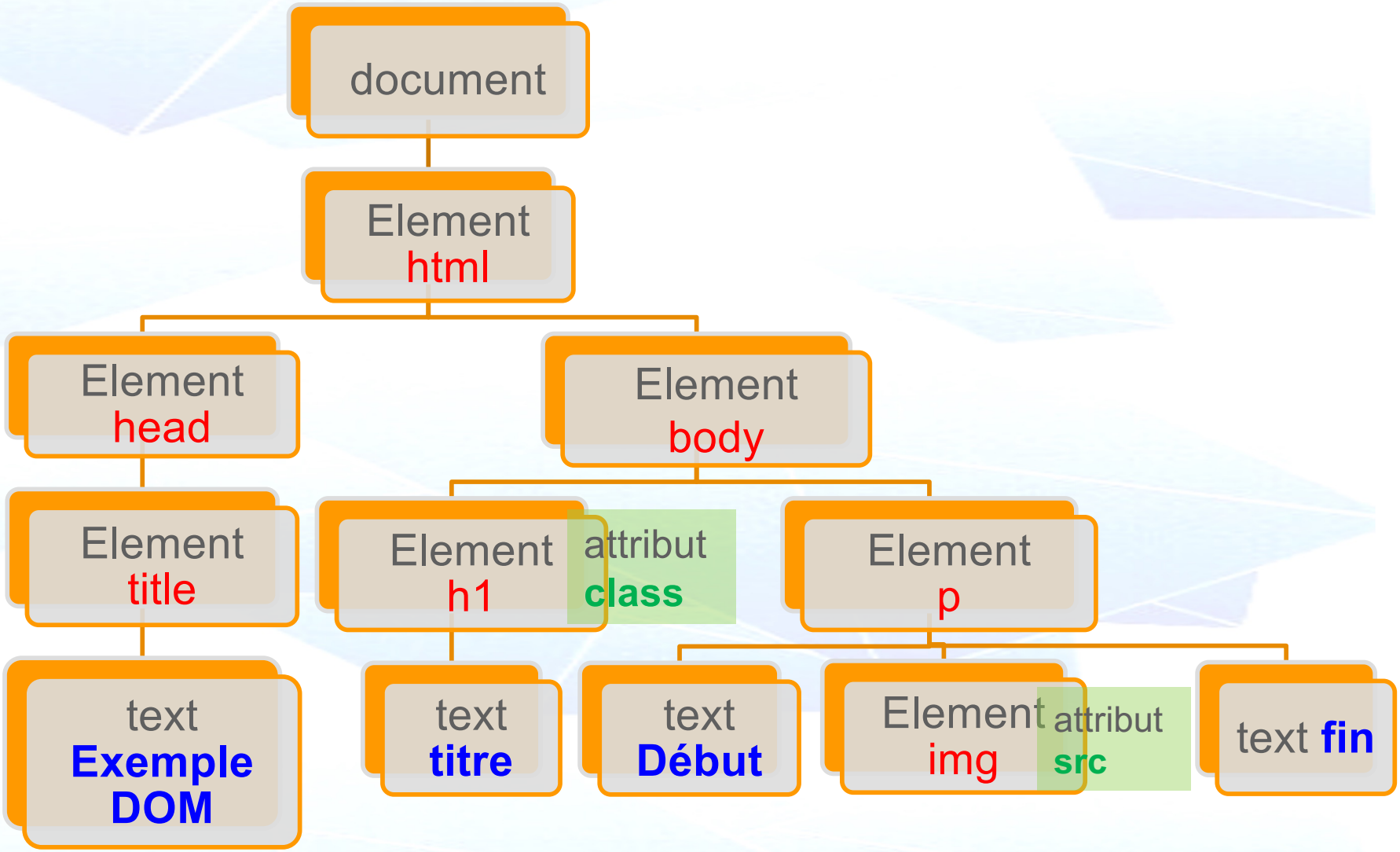
le DOM (Document Object Model)





le DOM *(Document Object Model)*





rechercher dans le DOM

Consultation du DOM en javascript

- rechercher un élément à partir de son ID *(attribut id HTML)*

```
<img id="img1" src="…" />
let img=document.getElementById("img1");
```

- rechercher des éléments à partir d'un nom de balise

```
let elems=document. getElementsByTagName("h1");
```

- rechercher des éléments à partir d'une classe

```
<div class="myclass" > … </div>
let s=document. getElementsByClassName("myclass");
```

rechercher dans le DOM

Consultation du DOM en javascript

- rechercher des éléments à partir d'un attribut HTML

```
<div name="toto" > … </div>
let s=document.getElementsByAttribute("name", "toto");
```

- rechercher des éléments à partir d'un sélecteur CSS

→ *le premier élement*

```
let elem=document.querySelector(".....");
```

→*tous les éléments*

```
let elems=document.querySelectorAll(".....");
```

rechercher dans le DOM

Consultation du DOM en javascript

let nœud = … ; // référence à un nœud du DOM

- liste des nœuds fils d'un nœud (y compris les nœuds textes et commentaires) : noeud.childNodes
- liste des nœuds fils d'un nœud (sans les nœuds textes et commentaires): noeud.children

- premier fils : noeud.firstChild
- liste des nœuds frères précédents un nœud : noeud.previsousSibling
- liste des nœuds frères suivants un nœud : noeud.nextSibling

rechercher dans le DOM

Consultation du DOM en javascript

let nœud = … ; // référence à un nœud du DOM

- nœud parent d'un nœud : noeud.parentNode
- type d'un nœud : noeud.nodeType

  // 1 pour ELEMENT_NODE // 3 pour TEXT_NODE
- nom d'un nœud : noeud.nodeName

rechercher dans le DOM

Consultation du DOM en javascript

let nœud = … ; // référence à un nœud du DOM

- premier (dernier) fils de type élément :
  noeud.firstElementChild
  noeud.lastElementChild
- liste des frères de type element
  noeud.previsousElementSibling
  noeud.nextElementSibling
- nœud parent de type element : noeud.parentNode

rechercher dans le DOM

```
<div id="tous">
  <div id="un">UN</div>
  " / "
  <div id="deux">DEUX</div>
  " \ "
  <div id="trois">TROIS</div>
  " ; "
</div>
```

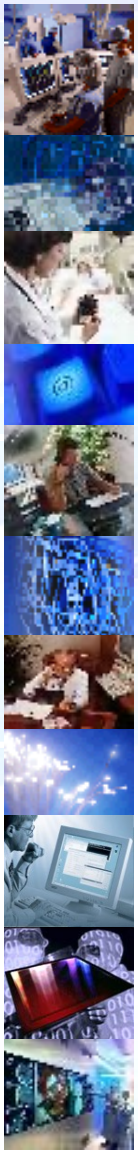
```
elt = document.getElementById("deux");
```

```
<div id="tous">
  <div id="un">UN</div>
  " / "
  <div id="deux">DEUX</div>
  " \ "
  <div id="trois">TROIS</div>
  " ; "
</div>
```

elt.nodeName → **div**

elt.nodeType → **1** (element)

rechercher dans le DOM



```
<div id="tous">
  <div id="un">UN</div>
  " / "
  <div id="deux">DEUX</div>
  " \ "
  <div id="trois">TROIS</div>
  " ; "
</div>
```

elt

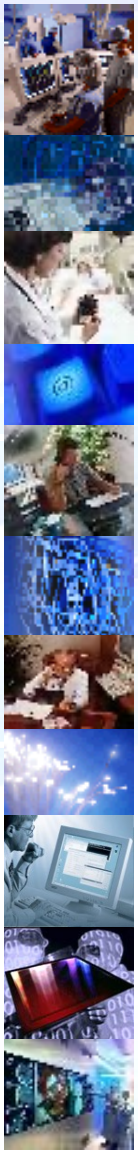
elt.parentNode

1

```
<div id="tous">
  <div id="un">UN</div>
  " / "
  <div id="deux">DEUX</div>
  " \ "
  <div id="trois">TROIS</div>
  " ; "
</div>
```

elt

rechercher dans le DOM



elt.previousSibling

2

```
<div id="tous">
  <div id="un">UN</div>
  " / "
  <div id="deux">DEUX</div>
  " \ "
  <div id="trois">TROIS</div>
  " ; "
</div>
```

elt

elt.nextSibling

3

```
<div id="tous">
  <div id="un">UN</div>
  " / "
  <div id="deux">DEUX</div>
  " \ "
  <div id="trois">TROIS</div>
  " ; "
</div>
```

elt

rechercher dans le DOM

elt.previousElementSibling

1

```
<div id="tous">
  <div id="un">UN</div>
  " / "
  <div id="deux">DEUX</div>
  " \ "
  <div id="trois">TROIS</div>
  " ; "
</div>
```

elt

elt.nextElementSibling

1

```
<div id="tous">
  <div id="un">UN</div>
  " / "
  <div id="deux">DEUX</div>
  " \ "
  <div id="trois">TROIS</div>
  " ; "
</div>
```

elt

rechercher dans le DOM

```
elt = document.getElementById("tous");
```

```
<div id="tous">
  <div id="un">UN</div>
  " / "
  <div id="deux">DEUX</div>
  " \ "
  <div id="trois">TROIS</div>
  " ; "
</div>
```

elt

```
<div id="tous">
  <div id="un">UN</div>
  " / "
  <div id="deux">DEUX</div>
  " \ "
  <div id="trois">TROIS</div>
  " ; "
</div>
```

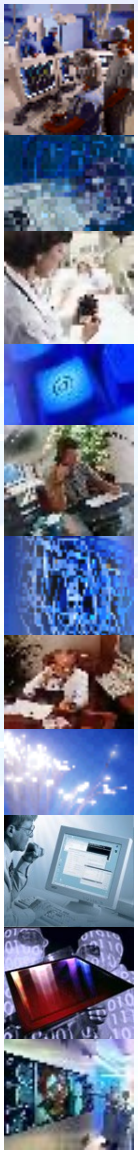
elt

```
elt.childNodes
```

```
elt.childNodes.length
```

6

rechercher dans le DOM



elt.children

```
<div id="tous">
  <div id="un">UN</div>
  " / "
  <div id="deux">DEUX</div>
  " \ "
  <div id="trois">TROIS</div>
  " ; "
</div>
```

elt

elt.children.length → 3

```
<div id="tous">
  <div id="un">UN</div>
  " / "
  <div id="deux">DEUX</div>
  " \ "
  <div id="trois">TROIS</div>
  " ; "
</div>
```

elt

elt.lastElementChild

elt.lastChild

consulter/modifier le DOM

Consultation/modification du DOM en javascript

let nœudElem = … ; // référence à un nœud du DOM
// de type element

- contenu HTML du nœud élément :
  noeudElem.innerHTML

  en lecture : txthtml = noeudElem.innerHTML
  en écriture : noeudElem.innerHTML = " abc "

- contenu textuel du nœud element et de ses descendants
  noeudElem.textContent (en lecture et écriture)

- style du nœud élément :
  noeudElem.style

  en lecture : st = noeudElem.style
  en écriture : noeudElem.style.color = "red"

modifier le DOM

Modification du DOM en javascript

- Créer un nœud élément : document.createElement()
- Créer un nœud texte : document.createTextNode()
- Créer un commentaire : document.createComment()

```
let elem = document.createElement("span")
```

➔ ce nœud n'est pas encore dans le DOM

- Ajouter un attribut à un élement :
  element.setAttribute(attr,val)

```
elem.setAttribute("name", "toto")
```

-> pour la modification de style, il est recommandé d'utiliser la propriété style

modifier le DOM

Modification du DOM en javascript

```
let nœudExistant =  …   ;   // référence à un nœud du DOM
let nœudNew =  …   ;   // nouveau nœud
```

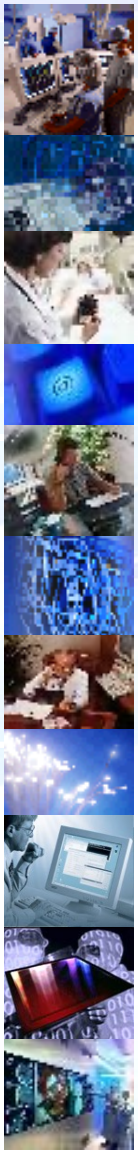
- Ajouter le nouveau nœud dans le DOM comme dernier fils d'un nœud existant :

```
noeudExistant.appendChild(noeudNew)
```

- Insérer le nouveau nœud parmi les fils d'un nœud existant :

```
// avant le 4ème fils par exemple
noeudExistant.insertBefore(noeudNew, list.childNodes[3])
```

modifier le DOM



Modification du DOM en javascript

- remplace un fils d'un nœud existant par un nouveau noeud

```
// remplacer le 2ème fils par exemple
noeudExistant.replaceChild(noeudNew, list.childNodes[1])
```

- supprimer un fils d'un nœud :

```
// ici le 4ème fils par exemple
noeudExistant.removeChild(list.childNodes[3])
```

modifier le DOM

```
<ul id="liste">
    <li>Confiture</li>
    <li>Beurre</li>
</ul>
```

Liste des courses

- Confiture
- Beurre

```
<ul id="liste">
    <li>Confiture</li>
    <li>Beurre</li>
    <li>Salade</li>
</ul>
```

Liste des courses

- Confiture
- Beurre
- Salade

```
// créer un nouveu produtit (un élément li)
let produit1 = document.createElement("li");
// créer un un nœud texte "salade"
let textProd1 = document.createTextNode("Salade")
// ajouter le texte comme fils du nœud li
produit1.appendChild(textProd1);
// sélectionner la liste à modifier
let liste = document.getElementById("liste");
// ajouter le nouveau produit à la liste
liste.appendChild(produit1);
```

modifier le DOM

```
<ul id="liste">
    <li>Confiture</li>
    <li>Beurre</li>
</ul>
```

Liste des courses

- Confiture
- Beurre

```
<ul id="liste">
    <li>Confiture</li>
    <li>Beurre</li>
    <li>Salade</li>
</ul>
```

Liste des courses

- Confiture
- Beurre
- Salade

```
// créer un nouveu produtit (un élément li)
let produit1 = document.createElement("li");
// définir le texte du produit avec "salade"
produit1.textContent = "Salade"
….
```

```
// modifier le contenu HTML de la liste
liste.innerHTML = liste.innerHTML + "<li>Salade</li>";
```

modifier le DOM

```
<ul id="liste">
    <li>Confiture</li>
    <li>Beurre</li>
    <li>Salade</li>
</ul>
```

Liste des courses

- Confiture
- Beurre
- Salade

```
<ul id="liste">
    <li>Confiture</li>
    <li>Salade</li>
</ul>
```

Liste des courses

- Confiture
- Salade

```
// supprimer le 2° fils de la liste
liste.removeChild(liste.children[1]);
```

modifier le DOM

```
<ul id="liste">
    <li>Confiture</li>
    <li>Salade</li>
</ul>
```

Liste des courses

- Confiture
- Salade

```
<ul id="liste">
    <li>Oranges</li>
    <li>Salade</li>
</ul>
```

Liste des courses

- Oranges
- Salade

```
// créer le produit <li>Oranges</li>
let produit2 = document.createElement(«li»);
let textProd2 = document.createTextNode("Oranges");
produit2.appendChild(textProd2);
// le mettre à la place du premier fils de la liste
liste.replaceChild(produit2,liste.children[0]);
```

```
// modifier le texte du premier fils de la liste
liste.children[0].textContent = "Oranges"
```

DOM et formulaires

Accéder à un formulaire ?

```
<form id="inscription">
   <input type="text" name="nom" id="nom"/>
   <input type="submit" id="sub" value="envoyer" />
</form>
```

- comme pour les autres éléments du DOM, par exemple si le formulaire a un id :

  let formu = document.getElementById("inscription")

- en utilisant les objets prédéfinis: l'objet document.forms contient l'ensemble des formulaires du document HTML

  document.forms[0] → premier formulaire
  document.forms["inscription"] → le form d'id inscription
  document.forms.inscription → le form d'id inscription

DOM et formulaires

Accéder à un champ du formulaire ?

```
<form id="inscription">
   <input type="text" name="nom" id="nom"/>
   <input type="submit" id="sub" value="envoyer" />
</form>
```

- comme un élément du DOM

  let nom = document.getElementById("nom")

- la propriété elements de l'objet forms contient l'ensemble des controles

  let formu = document.forms.inscription;
  formu.elements → tous les élements

  let nom = formu.elements.nom → l'input nom

DOM et formulaires

Accéder à un champ du formulaire ?

- la propriété value

let nom = document.getElementById("nom")

ou let nom = document.forms.elements.nom

Nom Durand envoyer

console.log(nom.value); → Durand

nom.value = "Dupont"; → Nom Dupont envoyer

Sur un bouton :

envoyer

document.getElementById("sub").value = "Confirmer";

Confirmer

DOM et formulaires

- les cases à cocher sélectionnées ?

☑ anglais ☐ espagnol ☑ allemand

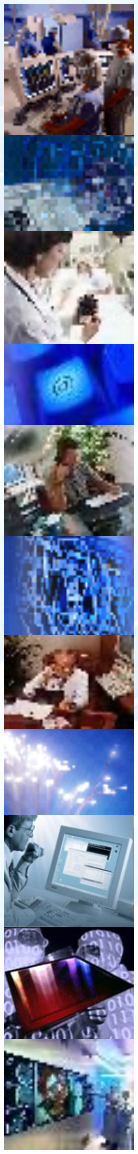
let nom = document.getElementById("nom")

```
// cases à cocher
let langues=document.getElementsByName('langue');
for (let i=0; i < langues.length; i++) {
        console.log(langues[i].value + " = "+ langues[i].checked)
}
```

```
anglais = true
espagnol = false
allemand = true
```

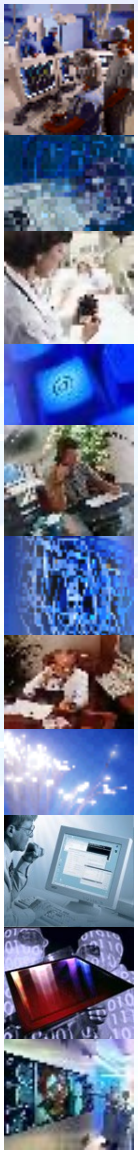
ISIS Ingénieur Santé numérique

UC Institut National Universitaire Champollion



Programmation événementielle

Programmation événementielle



Programmation événementielle ?

- dans ce paradigme de programmation, le programme n'est plus constitué d'une séquence d'instructions ; il est constitué de plusieurs bouts de code associés (souvent des fonctions) à des événements ; lorsque l'événement survient, la fonction est exécutée.
- Dans le cadre d'un navigateur web, il y a plusieurs catégories d'événements :
  - **actions de l'internaute** via le clavier et le souris
  - **autonome** : changement d'état, chargement de la page, de la taille de la fenêtre,…
  - **provenant des APIs** : animation, vidéo, SVG, drag and drop, …

Programmation événementielle

- événements window :
  - chargement : load unload
  - changement de page pageshow pagehide
- événements souris :
  - clic : click dblclick mousedown mouseup
  - déplacement : mouseover mouseenter mouseout
- événements clavier :
  - touches : keypress keydown keyup
- événements formulaires:
  - formulaire : submit reset
  - contrôle : change select input invalid

➔ liste complète *https://developer.mozilla.org/en-US/docs/Web/Events*

Programmation événementielle

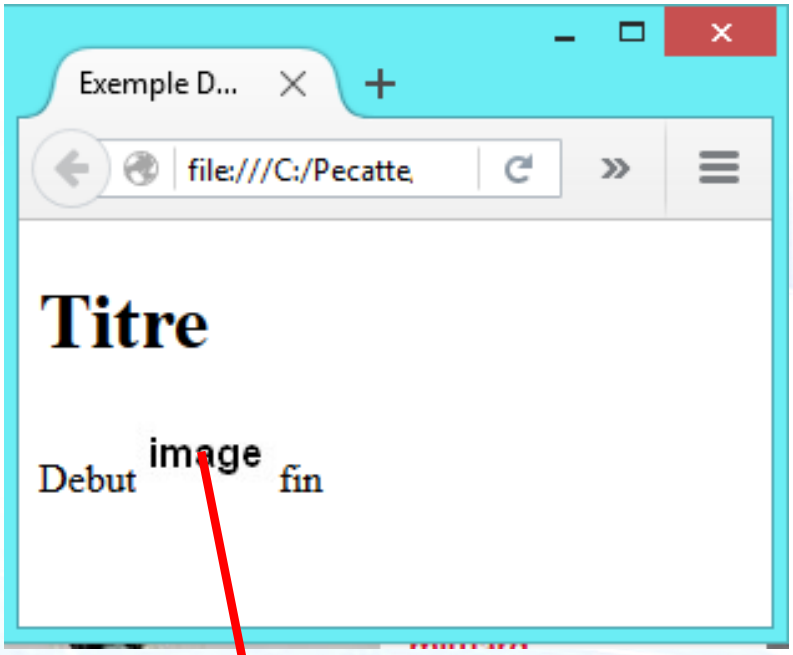
Prise en compte des événements dans un navigateur

Quels que soient les événements, ils suivent le MÊME modèle :

- l'événement est modélisé par un objet de type Event ➔ objet event
- l'objet event va jusqu'à l'élément cible de cet événement, en partant de la racine de l'arbre (capturing phase)
- s'il y a un traitement associé à cet événement pour cette cible, il est exécuté
- l'objet event remonte dans l'arbre (bubbling phase) et est traité par les nœuds qui écoutent cet événement

Programmation événementielle

```
<!DOCTYPE html>
<html>
<head>
    <title>Exemple DOM</title>
</head>
<body>
        <h1 class="titre">Titre</h1>
        <p>Début<img id="img1" src="img1.jpg"/>fin</p>
</body>
</html>
```



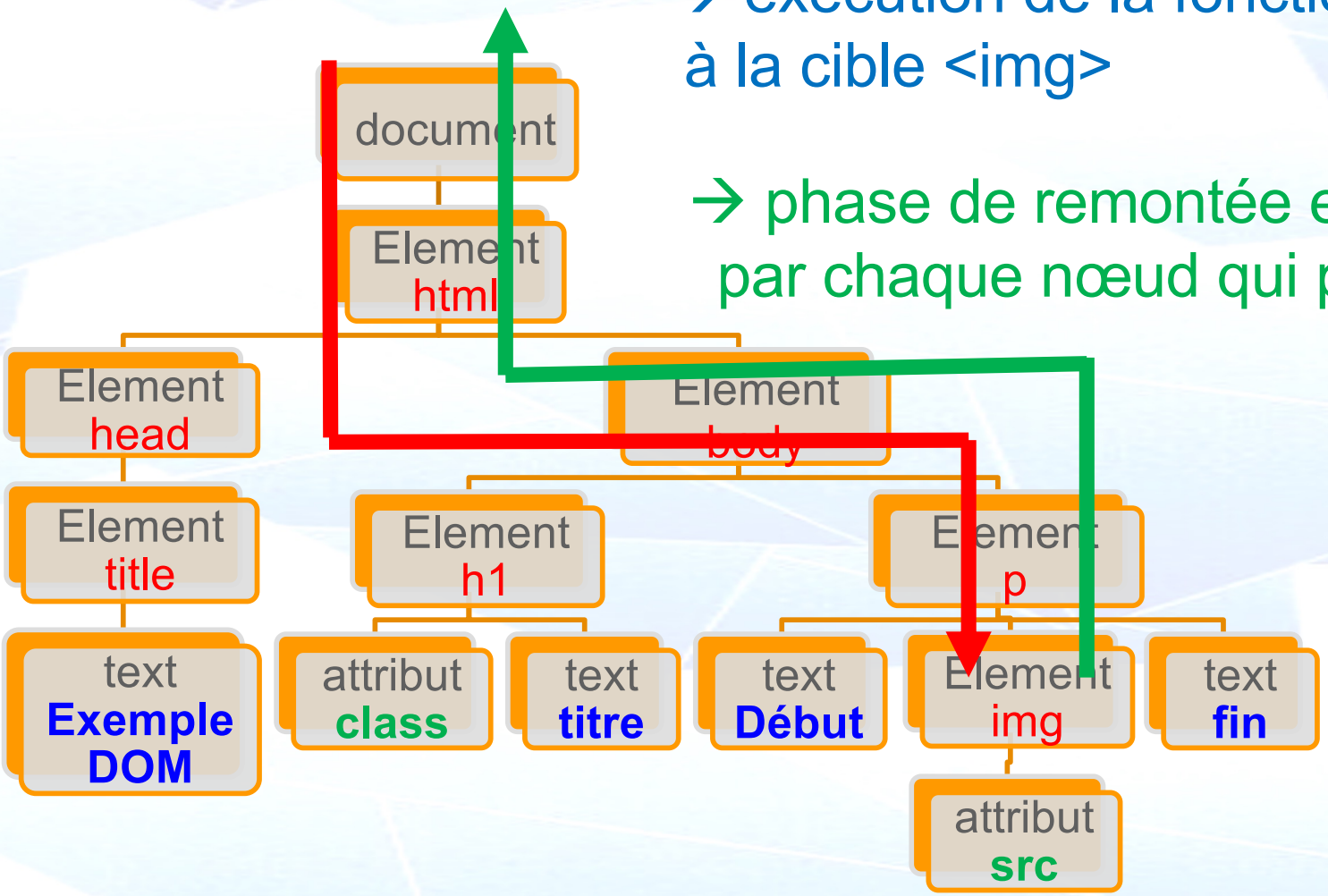
Afficher une popup lorsque l'utilisateur clique sur l'image

Programmation événementielle

Objet Event → phase de descente jusqu'à la cible <img>

→ exécution de la fonction associée à la cible <img>

→ phase de remontée en passant par chaque nœud qui peut réagir à l'evt



Programmation événementielle

Comment lier une fonction à un événement ?

- En javascript, plusieurs méthodes permettent de faire ce lien :
  - directement dans le code HTML à l'aide d'attributs spécifiques *(un attribut pour chaque événement)*

    <a href="…" **onclick**="alert('Bonjour !');">lien</a>

    ➔Mauvaise pratique : ne jamais l'utiliser !

  - directement dans le code Javascript avec une fonction spécifique addEventListener
    Celle-ci permet de mettre en place un écouteur d'événement et de lui indiquer la fonction à exécuter

Programmation événementielle

Comment associer un traitement à un événement donné pour une cible donnée ?

Il faut **identifier** 3 choses :

- le **type** *(cf liste dans précédente diapo)*
- la **cible** *(target)*
- l'**action associée** (sous forme d'une fonction)

et ensuite il faut **positionner un écouteur** (objet **listener**) sur la cible :

```
cible.addEventListener( typeEvenement, action ) ;
function action(event) {…}
```

→addEventListener est un exemple de fonction ayant une fonction en paramètre

Programmation événementielle

Exemple : affichage d'une popup si l'utilisateur clique sur l'image "img1"

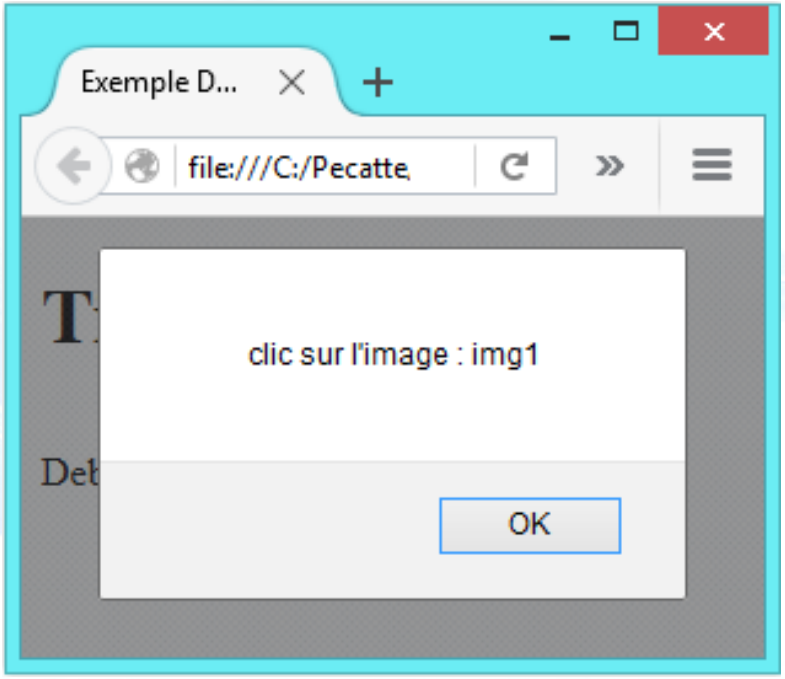
Type	'click'
Cible	L'élément d'id "img1"
Action	Afficher une popup

```
// une variable pour désigner la cible
let cible = document.getElementById("img1");
// la fonction qui réalise l'action
// elle a toujours comme paramètre l'objet event
function action(event) {
    alert("clic sur l'image : " + event.target.id);
}
```

Programmation événementielle

Exemple (suite)

```
// positionner un écouteur (un listener) sur la cible
cible.addEventListener('click', action);
```



Programmation événementielle

Même exemple → code plus synthétique

```
document.getElementById("img1").addEventListener('click',
        function action(event) {
                alert("clic sur l'image : " + event.target.id);
        }
);
```

cible événement

```
document.getElementById("img1").addEventListener('click',
        function action(event) {
                alert("clic sur l'image : " + event.target.id);
        }
);
```

fonction

Prog. ev. : l'objet Event

L'objet Event

Quand un événement se produit, toutes les informations le concernant sont enregistrées dans un objet appelé event.

- le type d'objet event varie selon l'événement.
- Quelques propriétés (selon les types d'événements) :
  - type : le type de l'événement,
  - clientX, clientY : coordonnées de l'événement par rapport au navigateur – l'écran – la page
  - key : information sur la touche appuyée
  - target : la cible de l'événement (== this)
  - …

https://developer.mozilla.org/fr/docs/Web/API/Event

Programmation événementielle

Il est parfois nécessaire de **stopper la phase de remontée** (bubbling phase)

```
event.stopPropagation()
```

De plus, il est aussi parfois nécessaire de **ne pas réaliser le comportement par défaut** associé à un évènement (par exemple, un clic sur un bouton submit lance la validation du formulaire et la redirection vers la page correspondant au lien.

```
event.preventDefault()
```

Prog.Ev : clavier

Exemple : popup affichant le code de la touche saisie

Type	'keypress'
Cible	Le document
Action	Afficher une popup avec le code de la touche

```
document.addEventListener('keypress',
            function action(event) { alert(event.key);  }
);
```

Prog.Ev : souris

Exemple : les coordonnées de la souris au clic sur les <p>

Type	'click'
Cible	Les éléments <p>
Action	Afficher une popup avec les coordonnées

plusieurs cibles mais addEventListener ne s'applique que sur une cible → il faut faire une boucle

```
let liste = document.getElementsByTagName("p");
for (let elt of liste) {
        elt.addEventListener('click', function action(event) {
                alert(event.clientX + event.clientY);
        });
}
```

Prog.Ev : formulaire

- Validation du formulaire → événement **submit**

  →lorsque l'utilisateur clique sur le bouton envoyer

```
<form id="inscription">
   <input type="text" name="nom" id="nom"/>
   <input type="submit" id="sub" value="envoyer" />
</form>
```

Dumoulin envoyer

```
let formu = ….;
formu.addEventListener('submit', function(event) {
  // référence à l'input
   console.log(document.getElementById("nom").value); → Dumoulin
// l'objet event
   console.log(event.target.nom.value); → Dumoulin
// l'objet this
   console.log(this.nom.value); → Dumoulin
});
```

Prog.Ev : formulaire

- Changement de valeur → événement **change**

  →par exemple choix d'une valeur dans une liste

```
<select name="dept" id="dept">
        <option value="chimie">Chimie</option>
        <option value="pec">PEC</option>
        <option value="mmi">MMI</option>
        <option value="gce">TC</option>
</select>
```

Département MMI
Chimie
PEC
MMI
TC

```
document.getElementById("dept").addEventListener('change',
  function(event) {
    console.log('valeur sélectionnée : ' + event.target.value);
})
```

→ mmi